

Architecture & Design Document

Personal Finance Tracker with AI Agent - Finn

Candidate:	Yogeshwara B
Problem Statement:	PS3 - Advanced
Live URL:	https://financetracker8.vercel
Document Date:	April 2026

1. Executive Summary

Finn is a full-stack personal finance tracker that lets authenticated users log, manage, and analyse expenses through a conversational interface. It supports two interaction modes: **Smart Mode** (LLM-driven, using Qwen2.5-72B-Instruct) and **Quick Mode** (rule-based slot-filling with NLP parsing). Both modes pre-populate profile fields and guide users through expense creation, viewing, modification, and analytics via a review-and-confirm step before any write operation.

The system is constrained to three expense categories (Food, Transport, Shopping) and two card types (Debit/Credit), enabling strong server-side and client-side validation. Analytics are rendered as Recharts bar charts directly inside the chat interface.

2. Technology Stack

2.1 Frontend

Technology	Version / Detail	Role
React + Vite	React 18, Vite build	UI framework, HMR development server
React Context + useReducer	Built-in React APIs	Global conversation state & slot management
Recharts	Chart library	Bar chart rendering for analytics views
Axios	HTTP client	All REST API calls to the backend
Supabase JS Client	Anon/publishable key only	Google OAuth, JWT session management
Vercel	Static hosting	CI/CD pipeline & edge deployment

2.2 Backend

Technology	Version / Detail	Role
Node.js + Express	Node 20, Express 5	REST API server, routing, middleware
Supabase (PostgreSQL)	Service role client	Primary relational database
HuggingFace Inference API	Qwen2.5-72B-Instruct	Smart Mode LLM inference
Render	Backend hosting	Node server deployment

3. Database Schema

Supabase (managed PostgreSQL) hosts two tables. Row Level Security (RLS) is enabled on **user_profiles** so each user can only access their own row. The **expenses** table is indexed on `contact_number` for fast per-user lookups.

3.1 expenses

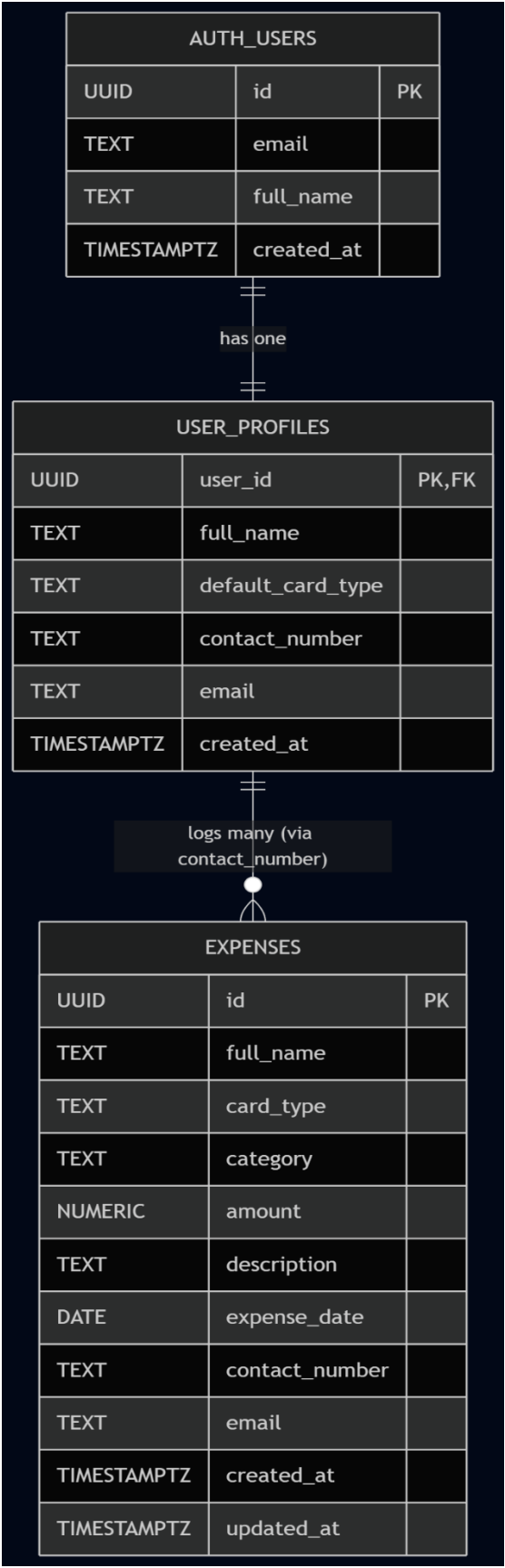
Column	Type	Constraints / Notes
id	UUID	PRIMARY KEY - DEFAULT <code>gen_random_uuid()</code>
full_name	TEXT	NOT NULL
card_type	TEXT	CHECK IN ('Debit Card', 'Credit Card')
category	TEXT	CHECK IN ('Transport', 'Shopping', 'Food')
amount	NUMERIC(12,2)	NOT NULL, CHECK > 0
description	TEXT	NOT NULL
expense_date	DATE	NOT NULL
contact_number	TEXT	NOT NULL - indexed: <code>idx_expenses_contact</code>
created_at	TIMESTAMPTZ	DEFAULT <code>now()</code>
updated_at	TIMESTAMPTZ	DEFAULT <code>now()</code> - auto-updated via trigger

3.2 user_profiles

Column	Type	Constraints / Notes
user_id	UUID	PRIMARY KEY - REFERENCES <code>auth.users(id)</code> ON DELETE CASCADE
full_name	TEXT	NOT NULL
default_card_type	TEXT	CHECK IN ('Debit Card', 'Credit Card')
contact_number	TEXT	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT <code>now()</code>

RLS policy: each authenticated user can SELECT/INSERT/UPDATE only rows where `user_id = auth.uid()`.

3.3 Entity Relationship Diagram



4. API Reference

Method	Endpoint	Auth	Description
POST	/api/expenses	—	Create expense (validated by middleware)
GET	/api/expenses?contact=	—	Fetch all expenses for a contact number
GET	/api/expenses/:id	—	Fetch single expense by UUID
PUT	/api/expenses/:id	—	Update expense date only
DELETE	/api/expenses/:id	—	Delete expense permanently
GET	/api/analytics?contact=&period=	—	Category totals, monthly totals, last 5
POST	/api/faq	—	FAQ keyword lookup
POST	/api/ai-parse	—	NLP regex field extraction
POST	/api/ai-chat	—	Full Qwen2.5-72B conversation turn
GET	/api/profile	JWT	Get user profile
POST	/api/profile	JWT	Upsert user profile
GET	/health	—	Health check

JWT auth uses Supabase auth.getUser() on the backend. All other endpoints are scoped by contact_number supplied by the frontend.

5.Validation Rules

All rules are enforced at two layers: the React frontend (UX feedback) and the Express middleware (enforcement). Both must pass before a record is written.

Field	Rule
full_name	Minimum 2 words required (first + last)
card_type	Exactly "Debit Card" or "Credit Card"
category	Exactly "Transport", "Shopping", or "Food"
amount	Positive number, max 2 decimal places
description	Non-empty string, max 300 characters
expense_date	DD-MM-YYYY format; must not be a future date
contact_number	Country code + exactly 10 digits — e.g. +911234567890

6. Application Modes

6.1 Smart Mode — LLM-Driven

Every user message is forwarded to **Qwen/Qwen2.5-72B-Instruct** via the HuggingFace Inference API router. The model returns a structured JSON object that drives all frontend state transitions.

Response schema

```
{ "reply": "string", "fields": {}, "intent": "create | view | modify | analytics | null" }
```

Modify action schema

```
{ "fields": { "action": "update_date | delete", "match_category": "Food", "match_position": 2, "new_date": "05-04-2026" } }
```

Parameter	Value / Behaviour
Model	Qwen/Qwen2.5-72B-Instruct via HuggingFace router
Temperature	0.2 - low for deterministic, structured JSON output
Max tokens	512 per response
Context window	Last 10 messages sent on every request
Intent routing	create · view · modify · analytics · null
Category inference	Semantic mapping: free text → Food / Transport / Shopping
Date resolution	"Today" → current date; default year 2026
Profile fields	Never requested - always sourced from user_profiles
Rate limit	30 AI calls per session (HuggingFace free tier)

6.2 Quick Mode — Rule-Based Slot Filling

Quick Mode uses **ConversationContext** (useReducer) as a slot registry. An NLP parser backed by regex extracts field values and assigns confidence scores.

Feature	Detail
Confidence scoring	Prevents overwriting a high-confidence slot with a weaker match
Amendment detection	Recognises "actually", "change", "I meant" - clears the targeted slot
Co-reference	Multiple fields extracted from a single user message
Profile pre-fill	Name, card type, phone loaded automatically at session start
Sequential prompting	Only asks for: category · amount · description · expense_date
NLP backend	POST /api/ai-parse - regex extraction with fallback scoring

7. Security Architecture

Control	Implementation
Secret management	No .env files committed - .env.example holds variable names only
Service role key	Backend-only; never bundled into the frontend
Frontend key	Supabase anon/publishable key - safe for client exposure
Row Level Security	user_profiles: users can only access rows where user_id = auth.uid()
CORS policy	Restricted to the known Vercel frontend origin
JWT validation	Profile endpoints call Supabase auth.getUser() on every request
Input validation	Dual-layer: React (UX) + Express middleware (enforcement)

8. End-to-End User Flow

Step	Stage	Detail
1	Authentication	User initiates Google OAuth via Supabase JS client
2	Session	JWT access token issued; Supabase session established in browser
3	Profile check	GET /api/profile - checked on every login
4	New user	Profile Setup form collects name, default card, phone
5	Profile save	POST /api/profile - upserted into user_profiles
6	Dashboard	Quick Mode / Smart Mode toggle presented
7	Create expense	Profile fields pre-filled; agent collects category, amount, description, date
8	Review card	All fields displayed with per-field Edit buttons
9	Confirm & save	POST /api/expenses - middleware validates → Supabase INSERT
10	View expenses	GET /api/expenses?contact= list embedded inline in chat
11	Modify / delete	Confirmation prompt → PUT or DELETE /api/expenses/:id
12	Analytics	GET /api/analytics → Recharts bar charts rendered inline

9. AI System Design

The Smart Mode AI pipeline is designed for deterministic, structured output. Low temperature (0.2) ensures the model consistently returns valid JSON. The system prompt enforces conversational reasoning rather than keyword matching, enabling natural field extraction and reliable intent classification across diverse phrasings.

System prompt strategy

Strategy	Behaviour
Intent detection	Conversational reasoning over the full message; not keyword matching
Category inference	Semantic mapping from free text - 'grabbed lunch' → Food
Date resolution	"Today" resolves to current system date; bare months default to 2026
Profile constraint	Never ask for name, card, or phone - always from user_profiles
Data integrity	Never fabricate expense data, never fake a save in the reply text
Output contract	Always respond with valid JSON: { reply, fields, intent }

Modify action JSON

When the user asks to change or delete an expense, the model populates the fields object with a structured action descriptor:

```
{ "fields": { "action": "update_date | delete", "match_category": "Food | Transport | Shopping",  
"match_position": 1, // 1-indexed from expense list "new_date": "DD-MM-YYYY" // only present for  
update_date } }
```

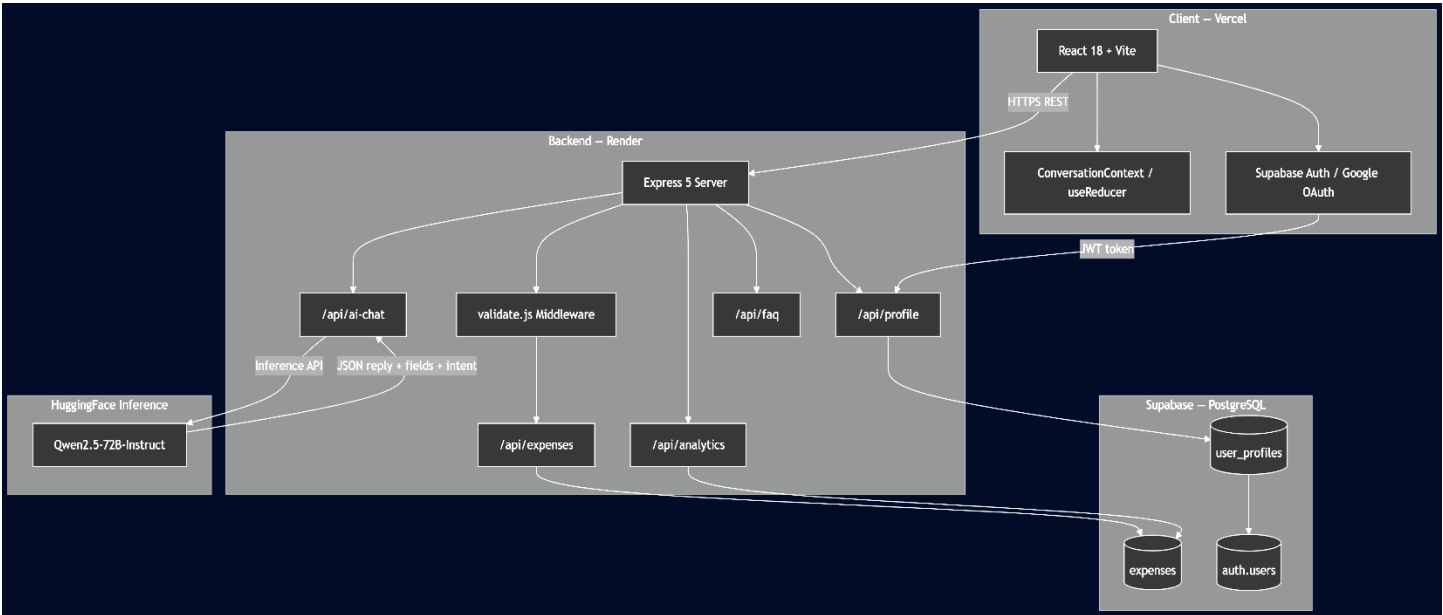
10. Known Limitations

Limitation	Detail
Partial modify support	Only date changes and deletes are supported. Category, amount, and description edits are not yet implemented.
AI rate limit	HuggingFace free tier imposes per-minute quotas. The app enforces a 30-call per-session ceiling in Smart Mode.
Cold start latency	Supabase free tier pauses after inactivity. First request after idle may take several seconds.
Single-user scope	Expenses are tied to one contact number per session. No multi-user sharing.
Category constraint	Only Food, Transport, and Shopping are supported. Adding a new category requires schema + validation changes.

11. System Architecture Overview

The application follows a three-tier architecture with a clear separation of concerns across presentation, application and data layers

Tier	Components	Communication
Presentation	React 18 SPA (Vercel) Supabase JS client (auth only)	Axios HTTP → Express REST API Supabase SDK → Google OAuth
Application	Node.js / Express 5 (Render) Validation middleware AI orchestration	Supabase service role → PostgreSQL HuggingFace API → Qwen2.5-72B
Data	Supabase (PostgreSQL) HuggingFace Inference API	Managed PostgreSQL (RLS enforced) Stateless LLM inference (per-request)



Authentication state is managed client-side via the Supabase JS SDK. The backend validates JWTs through `auth.getUser()` for protected profile endpoints. Expense endpoints are scoped by `contact_number` rather than per-request JWT validation, keeping them lightweight while maintaining data isolation.